Выполнил: Василенко И.А Группа: ПИа-090303-о24

ЛАБОРАТОРНАЯ РАБОТА №17

Дисциплина: Основы программирования на Python

Тема: Разработка асинхронного Telegram-бота на Python с использованием aiogram 3

Вариант 2: Библиотека

Цель работы

Изучить принципы разработки Telegram-ботов на Python с использованием асинхронного фреймворка aiogram 3 и получить практические навыки работы с Telegram Bot API, создания асинхронных обработчиков, использования asyncio, организации маршрутизации событий, применения Router и Dispatcher, обработки команд и текстовых сообщений, использования фильтров, создания Reply и Inline Keyboard, обработки Callback Query, применения конечного автомата состояний FSM для многошаговых диалогов, работы с middleware, валидации пользовательского ввода, хранения данных, взаимодействия бота с базой данных, обработки ошибок и разделения приложения на модули.

Постановка индивидуального задания (Вариант 2. Библиотека)

Разработать Telegram-бота библиотеки с сущностями книга, автор, читатель и выдача. Бот должен предоставлять каталог книг, поиск книг по названию и автору, выдачу книги читателю, возврат книги и список просроченных книг. Взаимодействие строится на Reply-клавиатуре главного меню и Inline-кнопках для действий с конкретными объектами, а многошаговые сценарии добавления книги и выдачи реализуются через конечный автомат состояний.

Краткие теоретические сведения

Telegram Bot API — HTTP-интерфейс, через который программа управляет ботом: получает обновления (сообщения, нажатия кнопок) и отправляет ответы. aiogram 3 — асинхронная библиотека для работы с этим интерфейсом, построенная на asyncio: обработчики объявляются как корутины и выполняются конкурентно, не блокируя друг друга.

Dispatcher получает обновления от Telegram и передаёт их в зарегистрированные Router. Router группирует обработчики по назначению и может включать другие роутеры, образуя дерево маршрутизации. Обработчик привязывается декоратором message() или callback_query() и сопровождается фильтрами: Command для команд вида /start, F для проверки текста кнопки, StateFilter для проверки текущего состояния.

Клавиатуры бывают двух видов. Reply Keyboard заменяет системную клавиатуру пользователя и подходит для постоянного меню, а Inline Keyboard прикрепляется к сообщению, и каждое нажатие передаёт в бот строку callback_data. Формат callback_data «действие:идентификатор», например lend:12, позволяет однозначно определить операцию и объект.

FSM (Finite State Machine) — механизм пошаговых диалогов. Состояния объявляются классами, наследующими StatesGroup, а текущее состояние и собранные данные хранятся в FSMContext. Middleware — промежуточный обработчик, выполняющийся до и после основного обработчика; он используется для журналирования событий и замеров времени. Данные хранятся в базе SQLite, работа с которой ведётся асинхронно через библиотеку aiosqlite.

Архитектура приложения

Приложение разделено на слои. Слой Telegram отвечает за приём событий и отправку сообщений, слой обработчиков содержит сценарии диалогов, слой клавиатур формирует разметку кнопок, слой сервисов содержит бизнес-правила

и форматирование текста, слой доступа к данным выполняет SQL-запросы, а слой состояний описывает шаги диалогов.

Компонент	Назначение
bot.py	Функция build_dispatcher(), подключение роутеров и middleware, запуск polling
config.py	Чтение токена бота и пути к базе из переменных окружения
handlers/start.py	Команды /start, /help, /cancel, вывод главного меню
handlers/menu.py	Каталог, карточка книги, поиск, мои книги, просроченные, возврат, изменение и удаление
handlers/registration.py	Многошаговые сценарии добавления книги и выдачи через FSM
keyboards/menu.py	Reply-клавиатура главного меню и inline-клавиатуры действий
states/book.py	Группы состояний AddBook и BorrowBook
middlewares/logging.py	Журналирование событий, счётчик сообщений, замер времени обработки
services/library_service.py	Бизнес-логика: добавление, поиск, выдача, возврат, просрочки
services/formatting.py	Формирование текстов карточек, каталога и списков выдач
database/	Асинхронное подключение aiosqlite, модели-структуры и репозиторий запросов
tests/	Двадцать восемь асинхронных тестов бизнес-логики и обработчиков

Сущности и база данных

Таблица	Поля	Связи
authors	id, full_name	Один автор — много книг
books	id, title, author_id, year, isbn, copies_total, copies_available	Ссылка на автора, много выдач
readers	id, telegram_id, full_name	Один читатель — много выдач
loans	id, book_id, reader_id, loan_date, due_date, return_date	Ссылки на книгу и читателя

Читатель связывается с Telegram-пользователем через поле telegram_id: при первом обращении бот создаёт запись или находит существующую, а в дальнейшем определяет читателя по идентификатору аккаунта. Схема базы создаётся при запуске функцией init_db(), что избавляет от отдельного шага миграции. Сущности описаны dataclass-структурами Book, Loan, Reader, Author, а доступ к данным инкапсулирован в репозитории, который выполняет параметризованные SQL-запросы и преобразует строки результата в объекты.

Router и обработчики

Обработчики разделены на три роутера по назначению: start отвечает за запуск и справку, menu — за навигацию по каталогу и операциям с книгами, registration — за многошаговые сценарии. Dispatcher подключает роутеры в порядке приоритета и передаёт каждому middleware.

Обработчик	Событие	Назначение
cmd_start	команда /start	Приветствие и вывод Reply-клавиатуры
cmd_help, cmd_cancel	/help, /cancel	Справка и сброс состояния диалога
cmd_catalog, cb_catalog	кнопка «Каталог»	Список книг с inline-кнопками
cb_book_detail	callback book:<id>	Карточка книги и действия с ней
cmd_find, cmd_search	/find, кнопка «Поиск»	Перевод в состояние ввода строки поиска
cmd_mybooks	кнопка «Мои книги»	Активные выдачи текущего читателя
cmd_overdue	кнопка «Просроченные»	Список просроченных выдач
cb_return, cmd_return	callback loan:<id>, /return	Возврат книги
add_book_start	кнопка «Добавить книгу»	Запуск FSM добавления книги
lend_start	/borrow	Запуск FSM выдачи книги
cb_delete, cb_confirm_delete	callback del:<id>	Удаление книги с подтверждением

Листинг 17.1 — Главное меню и маршрутизация каталога (handlers/menu.py)

```
@router.message(F.text == "📚 Каталог")
async def cmd_catalog(message: Message, state: FSMContext) -> None:
    await state.clear()
    books = await library_service.list_books()
    if not books:
        await message.answer("Каталог пуст.")
        return
    await message.answer(book_catalog_text(books),
                         reply_markup=books_kb(list(books)))


@router.callback_query(F.data.startswith("book:"))
async def cb_book_detail(callback: CallbackQuery) -> None:
    book_id = int(callback.data.split(":", 1)[1])
    book = await library_service.get_book(book_id)
    if book is None:
        await callback.answer("Книга не найдена", show_alert=True)
        return
    await callback.message.edit_text(format_book(book),
                                     reply_markup=book_detail_kb(book_id))
    await callback.answer()
```

Клавиатуры и Callback Query

Главное меню реализовано Reply-клавиатурой с семью пунктами: каталог, поиск, добавление книги, выдача книги, мои книги, просроченные и помощь. Кнопки постоянного меню всегда видны пользователю и не требуют обработки идентификатора.

Действия с конкретными объектами выполняются inline-кнопками, каждая из которых передаёт строку callback_data в формате «действие:идентификатор»: book:12 открывает карточку книги, lend:12 начинает выдачу, loan:7 возвращает выдачу, del:12 запускает удаление, edit:12 — изменение. После обработки вызывается callback.answer(), что снимает индикатор ожидания у пользователя. Клавиатуры каталога и выдачи формируются динамически по текущему списку книг и ограничены десятью кнопками, чтобы сообщение оставалось читаемым.

Листинг 17.2 — Формирование inline-клавиатуры (keyboards/menu.py)

```
def main_menu_kb() -> ReplyKeyboardMarkup:
    return ReplyKeyboardMarkup(
        keyboard=[
            [KeyboardButton(text="📚 Каталог"),
             KeyboardButton(text="🔍 Поиск")],
            [KeyboardButton(text="➕ Добавить книгу"),
             KeyboardButton(text="📖 Выдать книгу")],
            [KeyboardButton(text="↩️ Мои книги"),
             KeyboardButton(text="⏰ Просроченные")],
            [KeyboardButton(text="ℹ️ Помощь")],
        ],
        resize_keyboard=True,
    )


def books_kb(books: list[Book]) -> InlineKeyboardMarkup:
    kb = InlineKeyboardBuilder()
    for b in books[:10]:
        label = f"{b.title} — {b.author_name}"
        kb.button(text=label[:55], callback_data=f"book:{b.id}")
    kb.button(text="⬅️ Назад в меню", callback_data="menu")
    kb.adjust(1)
    return kb.as_markup()
```

Конечный автомат состояний

Многошаговые сценарии реализованы через FSM. Группа AddBook содержит шесть состояний: ввод названия, автора, года, ISBN, количества экземпляров и подтверждение. Группа BorrowBook содержит выбор книги,

подтверждение и выбор выдачи для возврата. На каждом шаге обработчик проверяет введённое значение, сохраняет его в FSMContext и переводит пользователя в следующее состояние; при ошибке валидации сообщение повторяется без изменения состояния.

После ввода всех данных бот показывает сводку и inline-кнопки подтверждения и отмены. Подтверждение создаёт запись в базе, сбрасывает состояние вызовом state.clear() и возвращает пользователя в меню. Команда /cancel прерывает любой сценарий и очищает накопленные данные, поэтому пользователь не может «застрять» в диалоге.

Листинг 17.3 — Состояния FSM (states/book.py)

```
from aiogram.fsm.state import State, StatesGroup

class AddBook(StatesGroup):
    title = State()
    author = State()
    year = State()
    isbn = State()
    copies = State()
    confirm = State()


class BorrowBook(StatesGroup):
    choose_book = State()
    confirm = State()
    return_choose = State()
```

Middleware и обработка ошибок

Собственный middleware LoggingMiddleware регистрируется на события сообщений и нажатий кнопок. Он определяет идентификатор пользователя, извлекает текст сообщения или строку callback_data, увеличивает счётчик сообщений пользователя и замеряет время обработки. При возникновении исключения в обработчике middleware записывает полную трассировку в журнал и пробрасывает ошибку дальше, не скрывая её от механизма обработки ошибок aiogram. Валидация пользовательского ввода выполняется в сервисном слое функциями normalize_year(), normalize_copies(), normalize_isbn() и validate_book_draft(), которые возбуждают LibraryError с понятным текстом, а обработчик перехватывает его и повторяет запрос.

Листинг 17.4 — Middleware журналирования (middlewares/logging.py)

```
class LoggingMiddleware(BaseMiddleware):
    def __init__(self, logger: logging.Logger | None = None) -> None:
        self._logger = logger or logging.getLogger("bot.middleware")
        self.message_counts: dict[int, int] = {}


    async def __call__(self, handler, event, data):
        started = time.monotonic()
        uid = None
        label = ""
        if hasattr(event, "from_user") and event.from_user is not None:
            uid = event.from_user.id
        if hasattr(event, "text") and event.text:
            label = event.text
        elif hasattr(event, "data") and event.data:
            label = f"callback:{event.data}"


        if uid is not None:
            self.message_counts[uid] = self.message_counts.get(uid, 0) + 1


        try:
            result = await handler(event, data)
        except Exception:
            self._logger.exception("Ошибка при обработке события %r", label)
            raise
        elapsed = (time.monotonic() - started) * 1000.0
        self._logger.info("uid=%s msg='%s' ok %.1f ms", uid, label, elapsed)
        return result
```

Раздел тестирования

Тесты написаны на основе pytest с поддержкой asyncio и не требуют подключения к Telegram: обработчики вызываются напрямую с объектами Message и CallbackQuery, а ответы перехватываются подменными объектами. База данных для каждого теста создаётся заново, что обеспечивает изоляцию.

Файл тестов	Кол-во	Что проверяется
tests/test_library.py	19	Создание, получение, изменение и удаление книги, поиск по названию и автору, пустой результат, валидация года, диапазона лет, количества экземпляров и ISBN, проверка черновика, выдача книги, отказ при отсутствии экземпляров, повторная выдача тому же читателю, возврат, просроченные выдачи, регистрация читателя, подсчёт объектов
tests/test_bot.py	10	Команда /start с выводом меню, отмена без активного состояния, полный FSM-сценарий добавления книги, подтверждение с созданием записи, выдача через FSM, поиск, «Мои книги» без выдач, возврат по callback, работа middleware и проброс исключения

Все двадцать девять тестов проходят успешно. Дополнительно бот проверялся вручную в Telegram: выполнялся запуск командой /start,

просматривался каталог, открывалась карточка книги, выполнялся поиск по автору, оформлялась выдача через пошаговый диалог, затем возврат книги, просматривались список просроченных и собственные выдачи. Отдельно проверялись отмена диалога и реакция на некорректный ввод года и количества экземпляров.

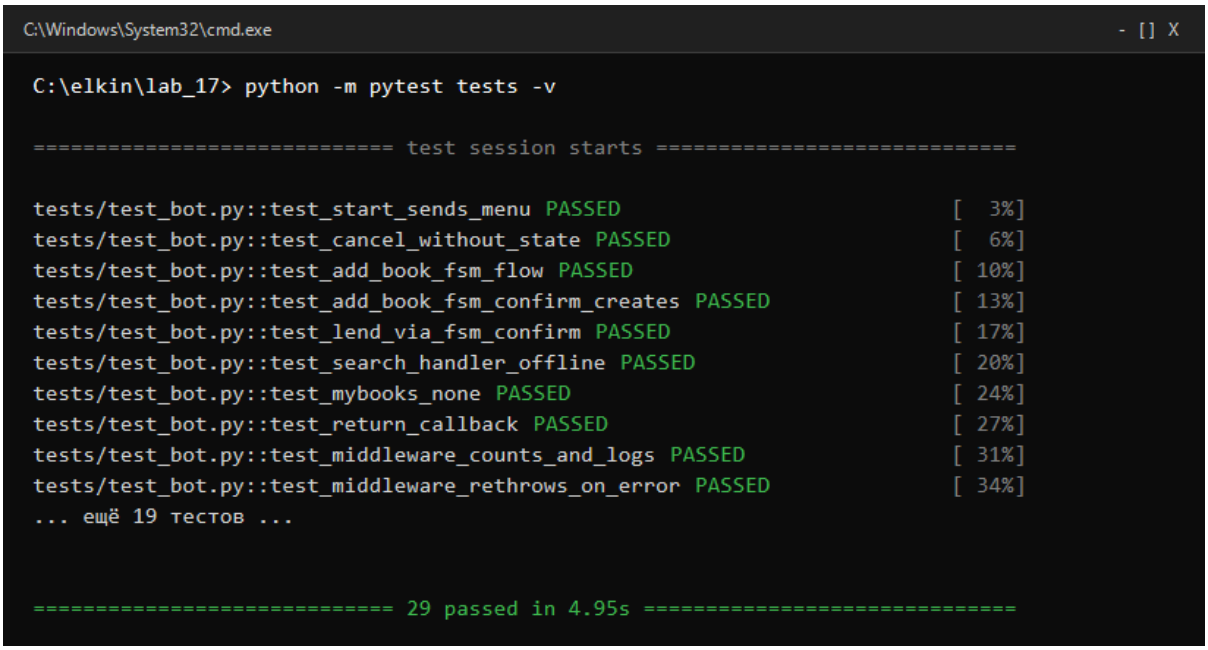


Рисунок 17.1 — Результат прохождения асинхронных тестов бота и бизнес-логики

Листинги ключевых фрагментов программы

Листинг 17.5 — Шаг FSM добавления книги (handlers/registration.py)

```
@router.message(AddBook.title)
async def add_book_title(message: Message, state: FSMContext) -> None:
    title = (message.text or "").strip()
    if len(title) < 2:
        await message.answer("Название слишком короткое. Повторите ввод:")
        return
    await state.update_data(title=title)
    await state.set_state(AddBook.author)
    await message.answer("Введите автора книги:")


@router.callback_query(AddBook.confirm, F.data == "add:confirm")
async def add_book_confirm(callback: CallbackQuery,
                           state: FSMContext) -> None:
    data = await state.get_data()
    book = await library_service.add_book(**data)
    await state.clear()
    await callback.message.edit_text(
        f"Книга «{book.title}» добавлена (id={book.id}).")
    await callback.answer("Готово")
```

Листинг 17.6 — Подключение роутеров (bot.py)

```
def build_dispatcher() -> tuple[Bot, Dispatcher]:
    bot = Bot(token=config.BOT_TOKEN)
    dp = Dispatcher(storage=MemoryStorage())


    dp.message.middleware(LoggingMiddleware())
    dp.callback_query.middleware(LoggingMiddleware())


    dp.include_router(start.router)
    dp.include_router(registration.router)
    dp.include_router(menu.router)
    return bot, dp


async def main() -> None:
    await init_db(config.DB_PATH)
    bot, dp = build_dispatcher()
    await dp.start_polling(bot)
```

Вывод

В ходе выполнения лабораторной работы разработан асинхронный Telegram-бот библиотеки на фреймворке aiogram 3. Освоены маршрутизация событий через Dispatcher и Router, обработка команд, текстовых сообщений и Callback Query, построение Reply- и Inline-клавиатур, а также организация многошаговых диалогов на основе конечного автомата состояний. Собственный middleware обеспечивает журналирование событий и замер времени обработки, валидация пользовательского ввода выполняется в сервисном слое, а данные хранятся в SQLite и читаются асинхронно. Разделение приложения на обработчики, клавиатуры, состояния, сервисы и репозиторий позволило покрыть бизнес-логику и сценарии двадцатью девятью автоматическими тестами без подключения к Telegram.